



Universidad Politécnica de Querétaro

Ingeniería en Sistemas computacionales

Materia: Programación web

“Documento de investigación – HU_1.5”

Docente: José Javier Moya Moya

Por: Andrés Joshua León Barranco, Alberto Luna Rufino, Eduardo Loredo García, Jorge Armando López Morales, Diego Jiménez Vargas y Saúl Alejandro Maldonado Ladrillero

Grupo: S-204

10/nov/2025

1. Objetivo

El objetivo es diseñar la estructura de base de datos necesaria para implementar un sistema de calificaciones y reseñas. Este sistema permitirá a los clientes calificar dos entidades distintas después de completar un pedido: el Restaurante (por la calidad de la comida) y el Repartidor (por la calidad de la entrega).

2. Análisis de requisitos de negocio

1. Autor: Solo un usuario con rol "Cliente" puede escribir una reseña.
2. Condición: Las reseñas solo pueden crearse para pedidos que ya tengan el estado "Entregado".
3. Cardinalidad: Un pedido (Pedido) solo puede tener una reseña (Review) asociada. Esto evita que un cliente califique el mismo pedido varias veces.
4. Dualidad: La reseña debe almacenar dos calificaciones independientes en un solo envío:
 - Calificación para el Restaurante (ej. 1-5 estrellas + comentario).
 - Calificación para el Repartidor (ej. 1-5 estrellas + comentario).

3. Propuesta de base de datos (Nueva tabla: reviews)

La solución más limpia y escalable es crear una nueva tabla llamada reviews que sirva como nexo entre el pedido, el cliente y las calificaciones.

Comando Artisan para crear la migración:

```
“php artisan make:migration create_reviews_table”
```

Estructura de la migración (..._create_reviews_table.php)

```
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
```

```

Schema::create('reviews', function (Blueprint $table) {
    $table->id();

    // --- Vínculos ---

    // Llave foránea al pedido que se está calificando.
    // Es 'unique' para asegurar 1 sola reseña por pedido.
    $table->foreignId('pedido_id')->unique()->constrained('pedidos')->onDelete('cascade');

    // Llave foránea al cliente que escribe la reseña.
    $table->foreignId('cliente_id')->constrained('users')->onDelete('cascade');

    // Llave foránea al restaurante que se está calificando.
    $table->foreignId('restaurante_id')->constrained('users')->onDelete('cascade');

    // Llave foránea al repartidor que se está calificando.
    // Nulable, por si el restaurante hace la entrega él mismo (a futuro).
    $table->foreignId('repartidor_id')->nullable()->constrained('users')->onDelete('set
null');

    // --- Calificación del Restaurante ---
    $table->unsignedTinyInteger('rating_restaurante'); // Almacena de 1 a 5
    $table->text('comentario_restaurante')->nullable();

    // --- Calificación del Repartidor ---
    $table->unsignedTinyInteger('rating_repartidor'); // Almacena de 1 a 5
    $table->text('comentario_repartidor')->nullable();

```

```

        $table->timestamps();
    });
}

public function down(): void
{
    Schema::dropIfExists('reviews');
}
};

```

4. Modificaciones a modelos Eloquent

Para que esta nueva tabla funcione, se deben actualizar los modelos existentes y crear uno nuevo.

1. Nuevo modelo: **App\Models\Review.php**

“php artisan make:model Review”

```
<?php
```

```
namespace App\Models;
```

```
use Illuminate\Database\Eloquent\Factories\HasFactory;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
class Review extends Model
```

```
{
```

```
    use HasFactory;
```

```
    protected $guarded = [];
```

```
    // Una reseña pertenece a un pedido
```

```
public function pedido()
{
    return $this->belongsTo(Pedido::class);
}

// Autor de la reseña
public function cliente()
{
    return $this->belongsTo(User::class, 'cliente_id');
}

// Restaurante calificado
public function restaurante()
{
    return $this->belongsTo(User::class, 'restaurante_id');
}

// Repartidor calificado
public function repartidor()
{
    return $this->belongsTo(User::class, 'repartidor_id');
}
}
```

3. Modificar App\Models\User.php

Añadir las relaciones para que un usuario (Restaurante o Repartidor) pueda "recibir" sus reseñas.

```
// ... (relaciones existentes)
```

```
// Reseñas que un usuario (Restaurante) ha recibido
```

```
public function reviewsRestaurante()
```

```
{
```

```
    return $this->hasMany(Review::class, 'restaurante_id');
```

```
}
```

```
// Reseñas que un usuario (Repartidor) ha recibido
```

```
public function reviewsRepartidor()
```

```
{
```

```
    return $this->hasMany(Review::class, 'repartidor_id');
```

```
}
```

```
// Reseñas que un usuario (Cliente) ha escrito
```

```
public function reviewsCliente()
```

```
{
```

```
    return $this->hasMany(Review::class, 'cliente_id');
```

```
}
```